

Natural Language Processing (NLP)

UNIT-1

Finding the Structure of Words

1. Words and their Components
2. Issues and Challenges
3. Morphological Models

1. Words and Their Components

1. Tokens
2. Lexemes
3. Morphemes
4. Typology

Tokens

- A token is a sequence of characters in a text that form a useful semantic unit.
- Tokenization is the process of breaking text into tokens.

Example: 'NLP is fun!'

Tokens: ['NLP', 'is', 'fun', '!']

Lexemes

- A lexeme is the base unit of meaning.
- It represents the abstract form of a word and all its inflected forms.
- Lexemes are often written in **uppercase or italics**
- Lexemes are important in **lemmatization**, where inflected forms are mapped to their root.

Example: Lexeme 'RUN'

Forms: ['run', 'runs', 'ran', 'running']

All these are considered forms of the same lexeme.

Morphemes

- A morpheme is the smallest meaningful unit in language.
 - Helps in morphological analysis, word formation, and understanding meaning.
 - Used in stemming and lemmatization.
 - Types of morphemes:
 - Free morphemes: can stand alone (e.g., 'book', 'run')
 - Bound morphemes: must attach to other morphemes (e.g., '-s', '-ing')
- Example: 'unhappiness' → ['un-', 'happy', '-ness']

Typology of Morphology

- Morphological typology classifies languages by their word formation processes.
- Major types of languages:
 - Isolating: Little or no inflection (e.g., Chinese)
 - Agglutinative: Simple affixes added (e.g., Turkish)
 - Fusional: Multiple meanings from single affix (e.g., Spanish)
 - Polysynthetic: Complex words that can express entire sentences (e.g., Inuit)

Isolating (Analytic) Languages

- **Definition:** Languages in which each word consists of a **single morpheme**.
- Words do **not** have inflections or affixes (prefixes, suffixes).
- **Grammatical relationships** are expressed through **word order** and **helper words** rather than inflection.

Examples:

- **Chinese:**
 - "Wǒ chī fàn" (I eat rice) → Each word = one morpheme
- Vietnamese
- **Characteristics:**
 - No or minimal morphology
 - Word order is crucial
 - Function words (e.g., "will", "is", "of") are commonly used

Agglutinative Languages

- **Definition:** Languages in which words are formed by **stringing together morphemes**, and **each morpheme has a distinct meaning**.
- Morphemes are clearly separable and don't change when joined.

Examples:

Turkish:

- *evlerinizden* = "from your houses"
 - *ev* (house) + *ler* (plural) + *iniz* (your) + *den* (from)

Finnish, Japanese, Swahili

Characteristics:

- Transparent morphology
- Easy to segment and analyze
- High regularity in affix use

Fusional (Inflectional) Languages

- **Definition:** Languages in which a **single morpheme encodes multiple grammatical features** (e.g., tense, number, case).
- The boundaries between morphemes are less clear compared to agglutinative languages.

Examples:

- **Spanish:**
 - *hablo* = "I speak"
 - root *habl-* (speak) + *-o* (first person singular, present tense)
- Latin, Russian, German

Characteristics:

- Morphemes may fuse several grammatical elements
- Complex internal changes (e.g., vowel shifts, consonant alternations)
- Less regular than agglutinative languages

Polysynthetic Languages

- **Definition:** Languages in which a **single word may include many morphemes**, expressing an entire sentence or complex idea.
- Extremely rich in morphology and often **noun incorporation** (nouns included in the verb).

Examples:

- **Inuktitut** (Eskimo-Aleut languages):
 - *qangatasuukkuvimmuuriaqalaaqtunga* = "I'll have to go to the airport"
- Mohawk, Chukchi, Yupik

Characteristics:

- Word = sentence
- Very complex word formation rules
- Heavy use of affixes and inflection

- Understanding morphological typology helps in:
 - **Tokenization and stemming**
 - **Designing language-specific NLP tools**
 - **Handling inflection-rich languages in translation**
 - Improving performance in **morphological analysis** and **machine translation**

2. Issues and Challenges

- Irregularity
- Ambiguity
- Productivity

Irregularity in Natural Language Processing (NLP)

Understanding the Challenges and
Solutions

What is Irregularity?

- Irregularity refers to non-uniform, unpredictable language patterns that break rules.
- These include irregular grammar, morphology, syntax, and semantics.
- Challenge: NLP models struggle with such inconsistencies, impacting performance.

Types of Irregularity

- 1. Morphological: go → went, run → ran
- 2. Phonological: rough /rʌf/, bough /baʊ/
- 3. Syntactic: Non-standard sentence structures
- 4. Semantic: Idioms like 'kick the bucket' (means to die)

Impact of Irregularity on NLP

- Machine Translation: Idioms are translated literally
- POS Tagging: Misclassification of irregular forms
- Parsing: Fails on non-standard sentence order
- Speech Recognition: Incorrect pronunciation mapping

Example

- Sentence: 'He ran fast, but he had gone unnoticed.'
- 'ran' (past of run) and 'gone' (past participle of go) are irregular
- Rule-based lemmatizers may fail to reduce them to 'run' and 'go'

Handling Irregularity

- Lexical Databases: Use of WordNet
- Annotated Corpora: Diverse training data
- Morphological Analyzers: Handle exceptions
- Neural Models: Transformers learn from context

Summary

- Irregularity causes deviations from language rules
- Impacts translation, parsing, and recognition
- Handled by data, models, and lexicons
- Essential to improve NLP accuracy

Ambiguity in Natural Language Processing (NLP)

Types, Examples, and Challenges

What is Ambiguity?

- Ambiguity occurs when a word, phrase, or sentence has more than one interpretation.
- It is a core challenge in NLP because machines struggle to infer meaning from context.

Types of Ambiguity

1. Lexical Ambiguity – One word, multiple meanings
2. Syntactic Ambiguity – Multiple parse trees
3. Semantic Ambiguity – Unclear sentence-level meaning
4. Pragmatic Ambiguity – Interpretation varies by context
5. Referential Ambiguity – Pronoun references unclear

Examples of Ambiguity

- Lexical: He went to the bank (river or financial?)
- Syntactic: I saw the man with the telescope
- Semantic: The chicken is ready to eat
- Pragmatic: Can you pass the salt?
- Referential: John hit Tom because he was angry

Why is Ambiguity Challenging in NLP?

- Context resolution is difficult for machines
- Dictionaries lack usage context
- Parsing generates multiple structures
- Disambiguation requires semantics and world knowledge

How NLP Handles Ambiguity

- Lexical – Word Sense Disambiguation
- Syntactic – Probabilistic/Dependency Parsing
- Semantic – Semantic Role Labeling, Contextual Embeddings
- Pragmatic – Dialogue Modeling
- Referential – Co-reference Resolution

Real-World NLP Models

- Transformers (BERT, GPT) capture context
- Sense2Vec: distinguishes word senses
- Tools: SpaCy, AllenNLP, CoreNLP
- Use embeddings and training context to resolve meaning

Summary

- Humans use context intuitively
- Machines require training, data, and modeling
- Ambiguity resolution is key to better NLP applications

Productivity in NLP

Understanding the generative power
of human language in computational
linguistics

What is Productivity?

- The capacity of language to generate new words, phrases, and sentences
- Uses a finite set of rules and vocabulary
- Challenges NLP systems to generalize to unseen forms

Why is Productivity Important?

- Language is creative and constantly evolving
- NLP systems must handle new, unseen data
- Affects morphology, syntax, and semantics

Examples of Productivity

- Morphology: 'teach' → 'teacher', 'talk' → 'talked'
- Compounding: 'smart' + 'watch' → 'smartwatch'
- Syntax: Recursive structures like nested clauses
- Lexical: New words like 'selfie', 'doomscrolling'

Challenges in NLP Due to Productivity

- Data sparsity and OOV (Out-of-Vocabulary) words
- Morphological complexity in unseen words
- Syntax generalization difficulties
- Ambiguity due to novel formations

Techniques to Handle Productivity

- Subword tokenization (e.g., BPE, WordPiece)
- Morphological analyzers (e.g., Morfessor)
- Character-level models for generalization
- Transfer learning with pre-trained LMs
- Data augmentation with synthetic examples

Summary

- Productivity enables infinite expression in language
- Essential for generalization and real-world applications
- Handling productivity improves robustness in NLP systems

Morphological Models

- Morphological Models refer to theoretical frameworks and computational approaches used to analyze, describe, or process the structure of words, especially how morphemes (the smallest meaning-bearing units) combine in various languages.

Morphological Models

- Used to understand the structure of the words.
- Models
 - Dictionary lookup
 - Finite state morphology
 - Unification based Morphology
 - Functional Morphology
 - Morphology Induction

Dictionary Lookup in NLP

- Understanding morphological analysis using dictionaries

- What is Dictionary Lookup?
- - Method for identifying word forms
- - Retrieves lexical information
- - Supports POS tagging, lemmatization

Why Use Dictionary Lookup?

- - Disambiguate meanings
- - Get grammatical features
- - Lemmatize inflections
- - Spell check and translation

Basic Components

- - Lexicon: known words
- - Morphological info: base and inflected forms
- - Rules for decomposition

Dictionary Lookup Models

- - Rule-Based Lookup
- - Table-based Lookup
- - ML Enhanced Lookup

Example – Custom Dictionary

- dictionary = {
 "run": {"POS": "verb", "definition": "to move swiftly"},
 "book": {"POS": "noun", "definition": "a set of pages"}
}
- Input: “run” → Output: verb, "to move swiftly"

Using NLP Libraries

- - NLTK: tokenize, POS tag, lemmatize
- - spaCy: tokenize, lemma, POS
- - WordNet: synonyms, definitions

spaCy Example

- `import spacy`
- `nlp = spacy.load("en_core_web_sm")`
- `doc = nlp("The cats are running.")`
- `for token in doc:`
- `print(token.text, token.lemma_, token.pos_)`

Challenges in Dictionary Lookup

- - Out-of-Vocabulary words
- - Ambiguity
- - Irregular forms
- - Slang and typos

Applications

- - Spell checking
- - Lemmatization
- - POS tagging
- - Translation
- - Chatbots

Dictionary lookup morphological model

- uses a list or database (a “dictionary”) that maps **each word form** (like *running, children, better*) to its **base form** (called the *lemma*, e.g., *run, child, good*), along with information like part of speech.
- It simply looks up words in this dictionary to normalize their forms.

How It Works?

- You receive a word like **“cats”**.
 - The system **searches the dictionary** for that exact word.
 - It finds that **“cats”** maps to **base form “cat”**, POS = noun.
 - It returns those properties—no guessing or statistical modeling needed.
-
- If a word is not in the dictionary, the model may:
 - Leave it unchanged,
 - Try simple rules (like dropping “-s”), or
 - Flag it as unknown.

- Great for languages like English with mostly regular word forms.
- Less effective for highly inflected languages (e.g., Turkish) unless combined with rules or fallback methods.
- Example
 - children → child (noun)
 - happier → happy (adjective)
 - singing → sing (verb)

Finite-State Morphological Model

- is a computer-based system that processes words using finite-state automata with input/output mappings.
- These systems are known as Finite-State Transducers (FSTs)—they take a surface word as input and output its morpheme structure (such as base form and grammatical features) or generate a form from underlying components.
- widely used for tasks like lemmatization, inflection, and morphological parsing across many languages

How Does an FST Work?

- It reads input characters (like “runs”). Each transition in the automaton can consume an input symbol and emit one or more output symbols—e.g., run+V+3SG for “runs.”
- By defining lexicon (list of stems) and rules for affixes or phonological changes, the FST models how words are generated and analyzed.
- The same transducer can be run either forward (analysis from word to analysis) or backward (generation).

- Input: The model takes a word like “foxes”.
- Transitions: The FST moves between states, changing input to output symbols:
 - Reads “f-o-x-e-s”
 - Outputs “fox + plural”
- Output: It returns the base form and any affixes or grammatical information.

- Consider English plural nouns:
- Lexicon of stems: cat, dog, fox
- Rules:
 - Add “-s” to form plural (e.g., "cat" → "cats")
 - If a word ends with a consonant + “y”, change “y” to “i” and add “-es” (e.g., “city” → “cities”)
- So the FST maps:
 - "cats" → "cat + plural"
 - "cities" → "city + plural"

Unification-Based Morphology

- What it is: Based on feature structures (attribute-value pairs) with unification (merging compatible features) to analyze word formation. Studocu
- How it works: Each morpheme has a structured feature description (e.g., lexeme: walk, number: plural). Combining affix and base uses unification. If features conflict, unification fails.
- Pros: Rich, expressive; can handle deep linguistic constraints and cross-linguistic variation.
- Cons: Computationally heavier, more complex to build.
- Use case: Complex languages (Arabic, Slavic, Persian) and grammar systems like HPSG.

Functional Morphology

- **What it is:** Focuses on the grammatical **function** of morphemes, like tense, number, part of speech. Often simpler and rule-oriented.
- **How it works:** Instead of full structural descriptions, mark functions: e.g., *cats* → plural noun, *running* → present participle verb.
- **Pros:** Simple, extendable, rule-based.
- **Cons:** Limited coverage—can't handle rich morphology well.
- **Use case:** Grammar-checking, basic POS systems, educational tools.

Morphology Induction

- **(Unsupervised Learning)**
- **What it is:** Learns morphological patterns automatically from raw text, without annotations. Finds stems, affixes, and rules from distribution patterns.
- **How it works:** Detect frequent endings/changes (e.g., “played,” “playing”), cluster forms, and infer rules like *-ed* for past tense.
- **Pros:** Doesn’t need annotated data; good for low-resource languages.
- **Cons:** Less accurate; may be noisy or over-generalize.
- **Use case:** Bootstrapping analyzers for new languages or domains.

Finding the Structure of Documents

Introduction

- Document structure discovery enables tasks like information retrieval, summarization, and question answering.
- The goal is to map large text into meaningful units, such as sentences and topic-based segments.

Why Is Document Structure Important?

- Foundations for text understanding, summarization, and retrieval.
- Enables parsing, question answering, and content navigation.
- Base requirement for advanced NLP tasks.

Main Methods Used

- Sentence Boundary Detection
- Topic Boundary Detection

- **Sentence Segmentation Example:**

- **Text sample:**

“Dr. Smith visited. He checked the lab.”

- Rule-based naive splitting would break after “Dr.” incorrectly.
- ML or neural methods can learn context and avoid false splits.

- **Topic Segmentation Example:**

- **Text sample:**

- Segment 1 (Tech news): “Latest smartphone release...”
- Segment 2 (Health): “New dietary guidelines...”
- Compute embedding similarity between adjacent sentence blocks; a big drop signals a topic boundary.

Sentence Boundary Detection

- Identifies sentence breaks (i.e. where one sentence ends and the next begins).
- Approaches include:
 - Rule-based methods (splitting at periods, question/exclamation marks, handling abbreviations).
 - Machine learning classifiers, including supervised models trained to detect boundaries using context features.
 - Neural sequence models (like LSTM, BERT) that are fine-tuned to predict sentence beginnings or endings

Topic Boundary Detection

- Marks points where the document's topic shifts.
- Techniques include:
 - **Lexical cohesion**: spotting changes in term frequency or use of key topic words.
 - **Similarity-based segmentation**: using cosine similarity across text segments, sharp drops often indicate a topic change.
 - **Clustering paragraphs or sentences** into topic groups and detecting transitions between clusters.
 - **Supervised classification**: binary models trained on labeled icon of a topic shift

Model Types Overview

- **Generative sequence classification:** models like HMMs or Naïve Bayes that estimate probabilities of transitions (boundary / no boundary) statistically.
- **Discriminative local sequence models:** classifiers or CRFs that label each boundary candidate using contextual features.
- **Sequence classification models:** aim to label sequences of sentences or tokens as boundaries or not.
- **Hybrid methods:** combine heuristics and statistical models for improved performance—e.g., rules + CRFs or neural models with post-processing rules

Complexity Analysis

- **Rule-based methods** are simple and efficient but may fail on ambiguous or noisy text.
- **Supervised/statistical methods** offer better accuracy but require training data.
- **Neural models** (transformers, RNNs) deliver high accuracy but are computationally heavier and require larger datasets.

Performance Factors

- Accuracy is typically evaluated using precision, recall, and F1-score on held-out labeled texts.
- Sentence segmentation is generally highly accurate; topic segmentation is more challenging due to its subjective nature.
- Real-world effectiveness depends on text domain (e.g., news, scientific articles, transcripts) and presence of markers like formatting or headings

Using NLTK's PunktSentenceTokenizer

- A classic generative approach where pre-trained models handle abbreviations and punctuation intelligently.

```
import nltk.data
text = """Mr. Smith bought a U.S.A. stamp. He loves reading books! Does he
read? Yes, he does."""
sent_detector = nltk.data.load('tokenizers/punkt/english.pickle')
sentences = sent_detector.tokenize(text.strip())
for i, s in enumerate(sentences, 1):
    print(f"{i}: {s}")
```

- NLTK's sentence tokenizer uses unsupervised models to distinguish periods in abbreviations from sentence endings.

Using **pySBD** (Rule-Based)

- PySBD (Pragmatic Sentence Boundary Disambiguation) provides robust rule-based detection across many languages.

```
import pysbd
```

```
text = "My name is Jonas E. Smith. Please turn to p. 55."
```

```
seg = pysbd.Segmenter(language="en", clean=False)
```

```
sentences = seg.segment(text)
```

```
print(sentences)
```

- pySBD handles tricky cases like abbreviations and punctuation gracefully.

Regex-Based Approach (Simple Heuristic)

- A manual method using basic Python regex for quick and flexible sentence splitting

```
import re
```

```
def split_sentences(text):
```

```
    return re.split(r'(?<=[.!?])\s+(?=[A-Z])', text)
```

```
text = "This is a sentence. And another? Yes!"
```

```
for idx, sent in enumerate(split_sentences(text), 1):
```

```
    print(f"{idx}: {sent}")
```

UNIT - II

- Syntax 1:
 - Parsing Natural language
 - Treebanks: A Data-Driven Approach to Syntax
 - Representation of Syntax Structure
 - Parsing Algorithms

- NLP -> enables the computer to understand the human language
- Each language has its own structure

↓
• Grammar

↓
Set of rules
E.g.: English -> SVO

She went for shopping (**correct**)
She shopping for went (**wrong**)

- So, natural languages follow some rules to generate sentences to express human opinions/views.

Parsing Natural language

- refers to the process of **analyzing and understanding the structure of a sentence** written or spoken in human language (like English, Hindi, etc.).
- It's a core task in **Natural Language Processing (NLP)**.
- Parsing breaks down a sentence into its parts, so a machine can understand:
 - **Grammar (syntax)** – how words fit together.
 - **Meaning (semantics)** – what the sentence actually means.

- There are two main strategies:
- **Top-Down Parsing** → Start from the root (Start Symbol S) and expand until you reach the input words.
- **Bottom-Up Parsing** → Start from the input words and combine (reduce) until you reach the root.

Top-Down Parsing

- **Process**

- Begin with **Start Symbol (S)**.
- Apply production rules to expand **non-terminals**.
- At each step, choose a rule that moves you closer to matching the input.
- Stop when the sequence of **terminals** matches the input sentence.

- **Example (The cat sleeps)**

Grammar:

$S \rightarrow NP VP$

$NP \rightarrow Det N$

$VP \rightarrow V$

$Det \rightarrow The$

$N \rightarrow cat$

$V \rightarrow sleeps$

Steps

1.S

2.NP VP

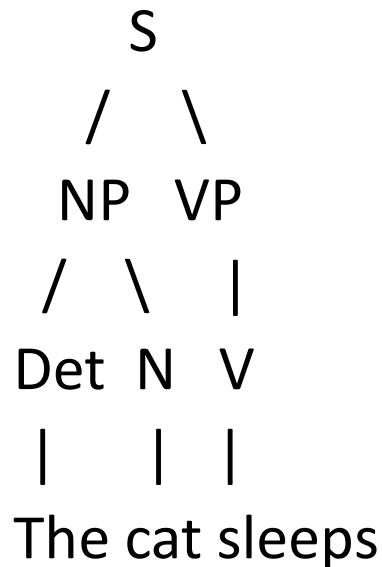
3.Det N VP

4.The N VP

5.The cat VP

6.The cat V

7.The cat sleeps



Bottom-Up Parsing

- Process
 - Begin with the input tokens.
 - Look for a right-hand side of a production rule in the current sequence and replace it with the left-hand side (reduction).
 - Repeat until the entire input reduces to the start symbol S .

- **Example (The cat sleeps)**

Grammar:

$S \rightarrow NP VP$

$NP \rightarrow Det N$

$VP \rightarrow V$

$Det \rightarrow The$

$N \rightarrow cat$

$V \rightarrow sleeps$

The cat sleeps

Det cat sleeps (The $\rightarrow$ Det)

Det N sleeps (cat $\rightarrow$ N)

NP sleeps (Det N $\rightarrow$ NP)

NP V (sleeps $\rightarrow$ V)

NP VP (V $\rightarrow$ VP)

S (NP VP $\rightarrow$ S)

The cat sleeps

↓ ↓ ↓

Det N V

\ / /

NP VP

\ /

S

Treebanks

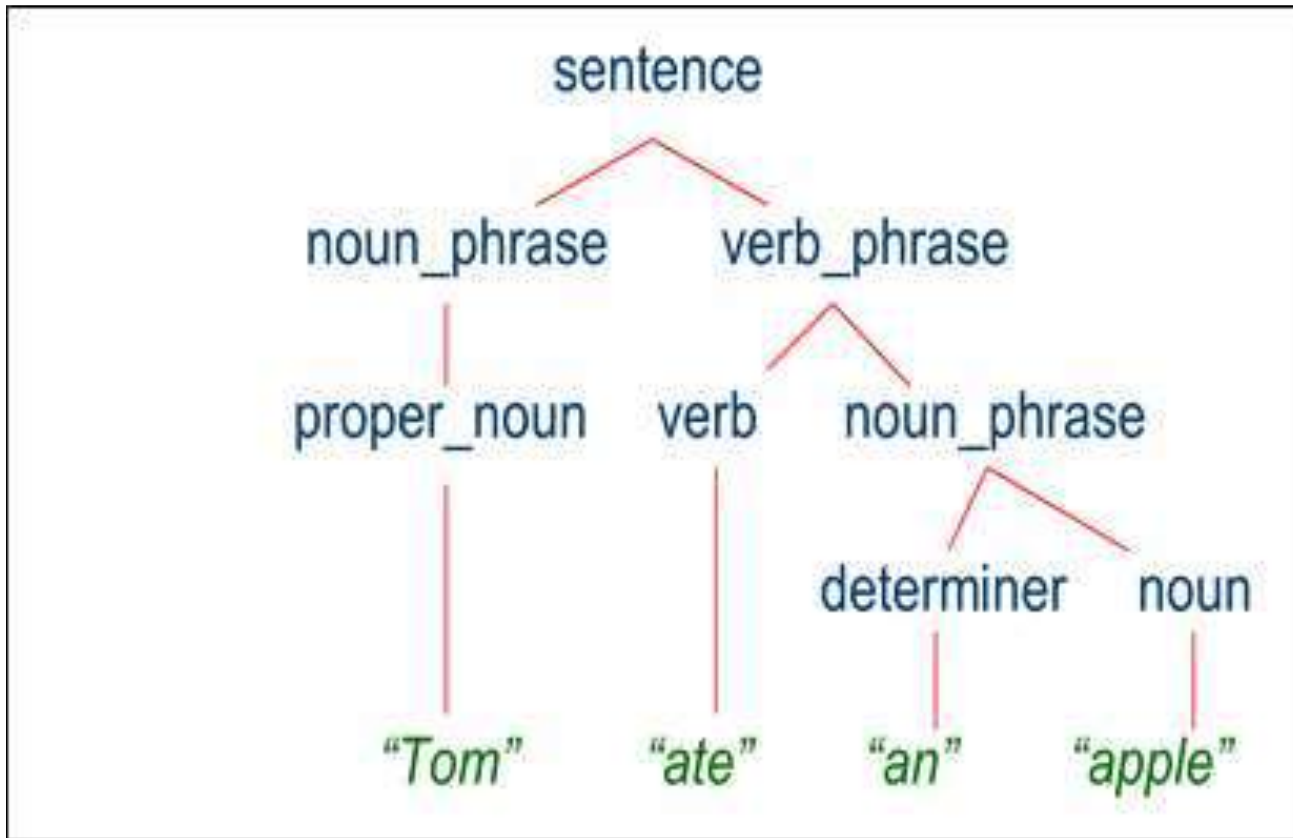
- A **treebank** is a linguistically annotated text corpus where each sentence is paired with its **syntactic** (and sometimes semantic) structure, usually in the form of a **tree**.
 - The “tree” shows how words group into phrases or how they depend on each other.
 - The “bank” refers to it being a large, stored collection of such trees.
- Treebanks are used for:
 - **Training parsers** in data-driven NLP (statistical or neural).
 - **Evaluating parser performance** by comparing predicted parses to gold-standard trees.
 - **Linguistic research** — studying grammar, syntax, and variation across languages.

- There are **two main types** of representation of structure:
 - Constituency Parsing tree
 - Dependency Parsing tree

1. Constituency Parsing

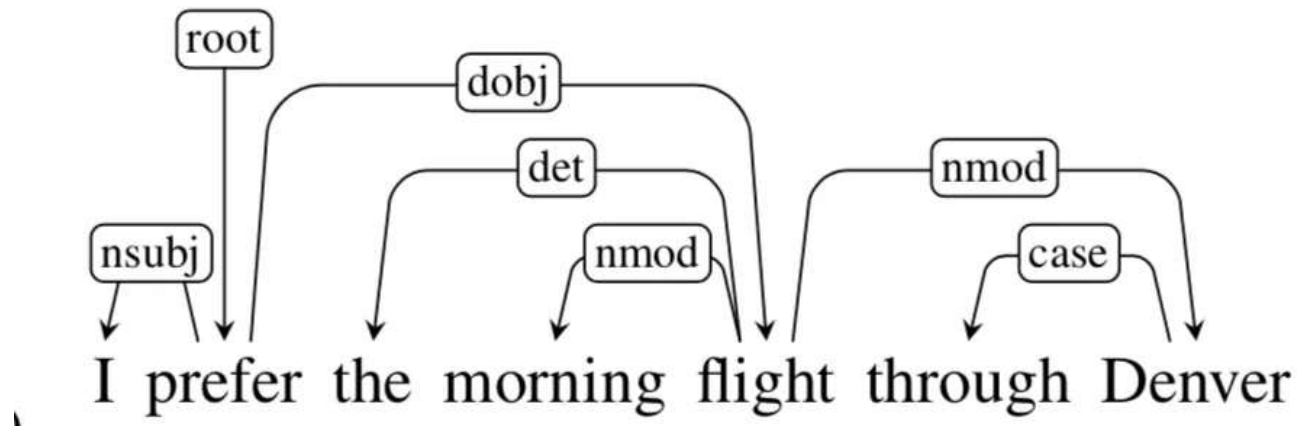
- Breaks sentences into nested sub-phrases (constituents).
- Based on phrase structure grammar (e.g., noun phrases, verb phrases).
- Output: a tree structure showing how words group into phrases.

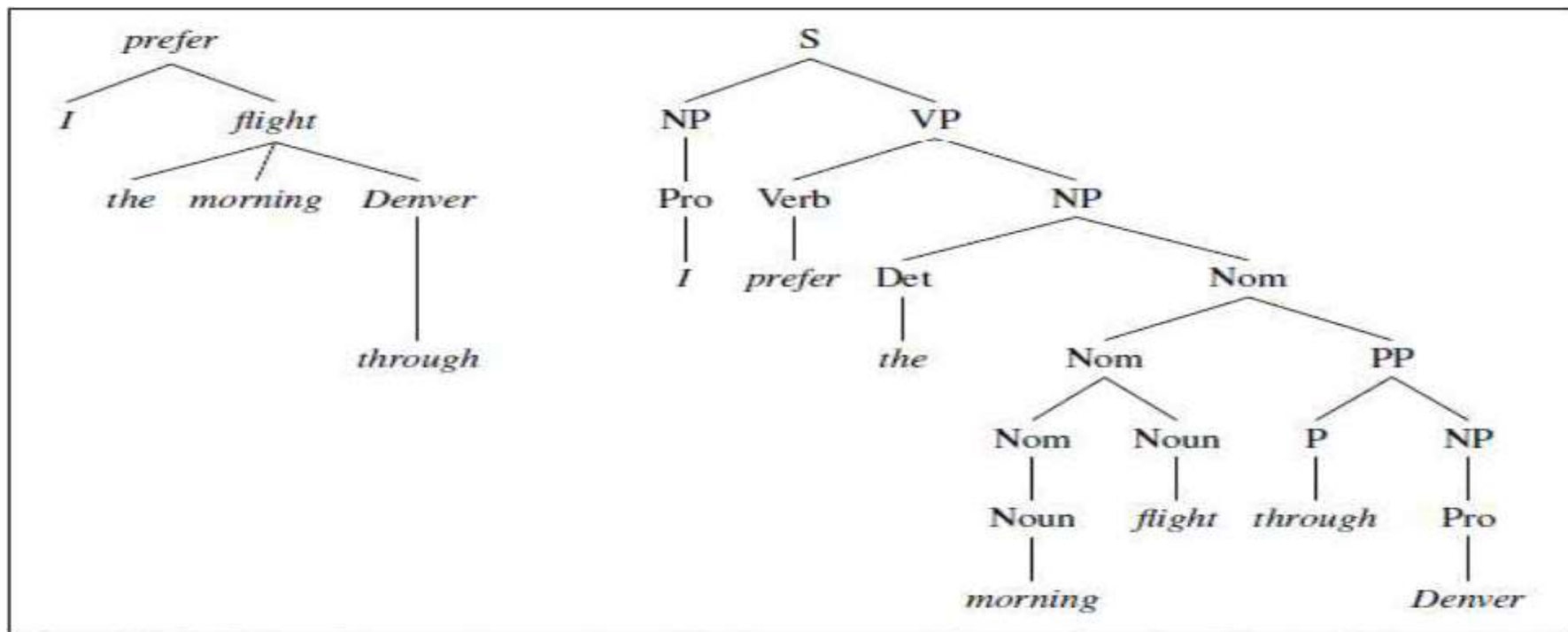
Example Sentence: "Tom ate an apple"



2. Dependency Parsing

- Focuses on **grammatical relationships** between words.
- Represents the sentence as a graph:
 - words are nodes,
 - dependencies are edges.
- Eg: Sentence: “ I prefer the morning flight through Denver”



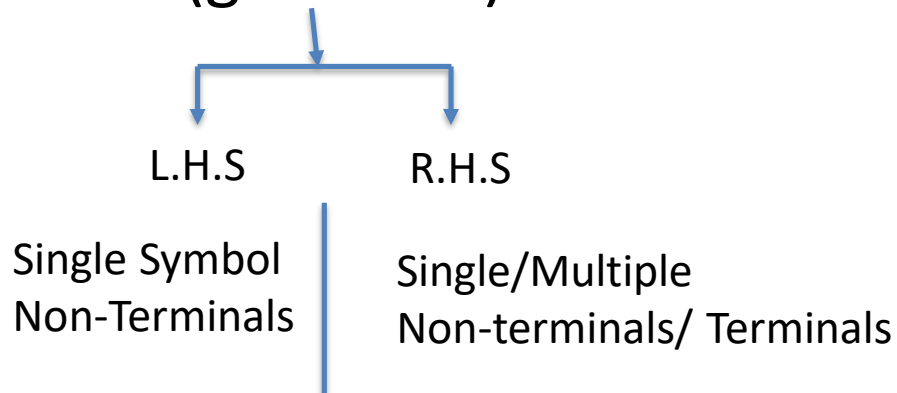


Sentence: "I prefer the morning flight through Denver"

Constituency Parsing

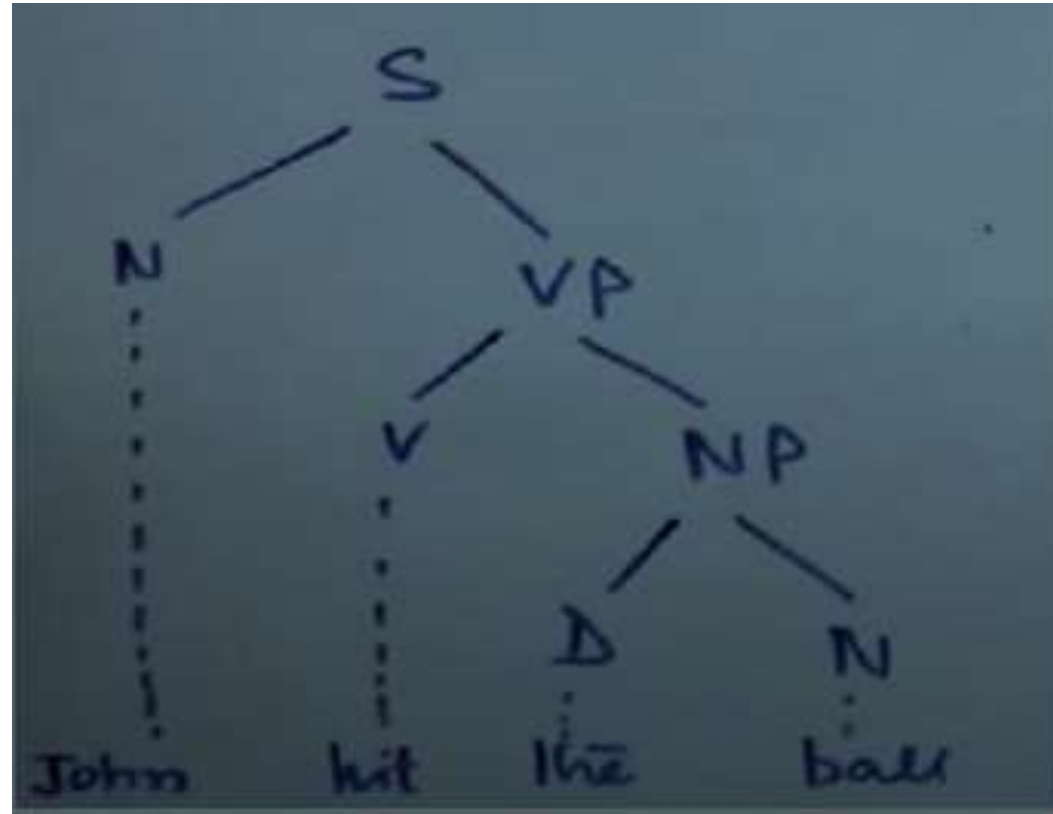
- CFL (Context-Free Language)
 - is a language generated by a **Context-Free Grammar (CFG)**. It is a type of formal language that can describe the syntax of programming languages and many aspects of natural languages.
- A CFG consists of:
 - Terminals (words/symbols): actual words in a sentence (e.g., cat, sat)
 - Non-terminals: syntactic categories (e.g., NP, VP, S)
 - Start symbol: typically S (sentence)
 - Production rules: define how non-terminals can be replaced with other non-terminals or terminals.

- CFG $\rightarrow$ Is a formal grammar used to generate all possible strings in a given formal language.
- $G(V, T, P, S)$
 - $V \rightarrow$ Variables/Non-terminals
 - $T \rightarrow$ Terminals
 - $P \rightarrow$ Production rules
 - $S \rightarrow$ Start symbol
 - Language $\rightarrow$ set of rules (grammar)



Example: John hit the ball

- $S \rightarrow NP, VP$
- $VP \rightarrow V, NP$
- $NP \rightarrow D, N$
- $V \rightarrow \text{hit}$
- $D \rightarrow \text{the}$
- $N \rightarrow \text{John} \mid \text{ball}$

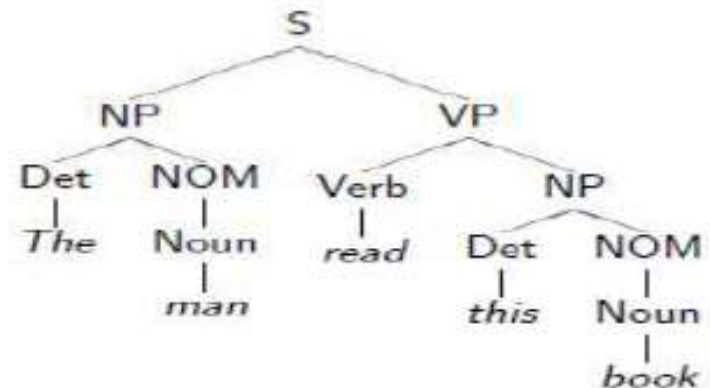


Application of grammar rewrite rules

$S \rightarrow NP VP$	$Det \rightarrow that \mid this \mid a \mid the$
$S \rightarrow Aux NP VP$	$Noun \rightarrow book \mid flight \mid meal \mid man$
$S \rightarrow VP$	$Verb \rightarrow book \mid include \mid read$
$NP \rightarrow Det NOM$	$Aux \rightarrow does$
$NOM \rightarrow Noun$	
$NOM \rightarrow Noun NOM$	
$VP \rightarrow Verb$	
$VP \rightarrow Verb NP$	

$S \rightarrow NP VP$
 $\rightarrow Det NOM VP$
 $\rightarrow The NOM VP$
 $\rightarrow The Noun VP$
 $\rightarrow The man VP$
 $\rightarrow The man Verb NP$
 $\rightarrow The man read NP$
 $\rightarrow The man read Det NOM$
 $\rightarrow The man read this NOM$
 $\rightarrow The man read this Noun$
 $\rightarrow The man read this book$

Parse tree



Parsing algorithms

- Parsing in NLP refers to the process of analyzing a sentence or text to determine its grammatical structure and the relationships between its components.
- Parsing algorithms:
 - Shift-Reduce Parsing
 - Hyper Graph & Chart Parsing
 - Minimum Spanning Tree
 - Dependency Parsing

Shift-Reduce Parsing

- Shift–reduce parsing is a **bottom-up parsing technique** used in NLP and compiler design to build a parse tree for a sentence (string).
- **Bottom-up** means it starts from the words (tokens) and tries to **reduce** them to the start symbol of the grammar.
- The parser uses two data structures:
 - **Stack** → holds symbols (terminals/non-terminals)
 - **Input buffer** → holds the remaining input tokens
- The parser performs actions until the input is fully parsed.

- **Four Possible Actions**

- **Shift:** Move the next input token to the top of the stack.
- **Reduce:** Replace a sequence of symbols on the stack that matches the RHS of a grammar rule with the LHS (reduce by rule).
- **Accept:** If the stack contains the start symbol and input is empty $\rightarrow$ successful parse.
- **Error:** If no valid move exists.

- **Example 1** - Consider the grammar

$$S \rightarrow S + S$$

$$S \rightarrow S * S$$

$$S \rightarrow id$$

- Perform Shift Reduce parsing for input string "id + id + id".

Stack	Input Buffer	Parsing Action
\$	id+id+id\$	Shift
\$id	+id+id\$	Reduce S->id
\$S	+id+id\$	Shift
\$S+	id+id\$	Shift
\$S+id	+id\$	Reduce S->id
\$S+S	+id\$	Reduce S->S+S
\$S	+id\$	Shift
\$S+	id\$	Shift
\$S+id	\$	Reduce S->id
\$S+S	\$	Reduce S->S+S
\$S	\$	Accept

- **Example 2** - Consider the grammar

$$E \rightarrow 2E2$$

$$E \rightarrow 3E3$$

$$E \rightarrow 4$$

Perform Shift Reduce parsing for input string "32423".

Stack	Input Buffer	Parsing Action
\$	32423\$	Shift
\$3	2423\$	Shift
\$32	423\$	Shift
\$324	23\$	Reduce by $E \rightarrow 4$
\$32E	23\$	Shift
\$32E2	3\$	Reduce by $E \rightarrow 2E2$
\$3E	3\$	Shift
\$3E3	\$	Reduce by $E \rightarrow 3E3$
\$E	\$	Accept

- **Try with below grammar**

- Consider the grammar

$$S \rightarrow (L) \mid a$$
$$L \rightarrow L , S \mid S$$

Perform

Shift Reduce parsing for input string "(a, (a, a))".

- Consider the Grammar:

$$E \rightarrow E + T \mid T$$
$$T \rightarrow T * F \mid F$$
$$F \rightarrow \text{id}$$

Perform Shift Reduce parsing for the input string "id + id + id".

Chart Parsing

- **CYK Parsing** (Cocke–Younger–Kasami parsing)
- CYK is a bottom-up parsing algorithm used for context-free grammars (CFGs).
- It works only if the grammar is in Chomsky Normal Form (CNF) (where every production is either $A \rightarrow BC$ or $A \rightarrow a$).
- The algorithm uses dynamic programming to check whether a string can be generated by the grammar, and optionally construct a parse tree.

A ->a (correct)

A ->B | C (correct)

NP -> the | N (**Wrong**)

NP -> Det | N | PP (**wrong**)

NP-> the | N

NP -> Det | N (..introduce a dummy non-terminal to original terminal)

Det ->the

NP -> Det | N | PP

Nominal -> N | PP

N -> Det | Nominal

- Given grammar
 - $S \rightarrow NP \mid VP$
 - $NP \rightarrow Det \mid N$
 - $VP \rightarrow V \mid NP$
 - $V \rightarrow include$
 - $Det \rightarrow the$
 - $Det \rightarrow a$
 - $N \rightarrow meal$
 - $N \rightarrow flight$
- Sentence. “the flight includes a meal”

Sentence: the flight includes a meal

0 the 1 flight 2 includes 3 a 4 meal 5

	1	2	3	4	5
0	Det ←	NP ←			S
1		N ↓			
2			V ←		VP
3				Det ←	NP
4					N ↓

- Given grammar
 - (0.3) $S \rightarrow NP \mid VP$
 - (0.3) $NP \rightarrow Det \mid N$
 - (0.2) $VP \rightarrow V \mid NP$
 - (0.05) $V \rightarrow \text{include}$
 - (0.4) $Det \rightarrow \text{the}$
 - (0.4) $Det \rightarrow \text{a}$
 - (0.01) $N \rightarrow \text{meal}$
 - (0.02) $N \rightarrow \text{flight}$
- Sentence. “the flight includes a meal”

NP (0.3)

/ \

Det(0.4) N(0.02) = 0.024

	1	2	3	4	5
0	Det 0.4	NP ↓			S ↓
1		N 0.02			VP ↓
2			V 0.05		NP ↓
3				Det 0.4	N 0.01
4					

